Architectures and Infrastructures for Artificial Intelligence

Practice 1: Cloud Computing Services

Tamara Besedina

October 2026

1 Objective

Learn how to offload AI processing to the cloud to observe the advantages of Cloud Computing.

2 System overview

The project implements a simplified Smart Lighting System in which public webcams act as presence sensors. Frames are periodically captured from fixed cameras overlooking public areas. The frames are analysed to identify people and vehicles that could determine whether lighting should be activated in the corresponding control zone.

The workflow:

1. local capture and local YOLOv8 inference;
2. frame capture on the local computer and inference on an Azure virtual machine;
3. serverless processing in Azure;
4. extension of both cloud architectures to multiple cameras.

2.1 Project repository

The source code is available in the cloud-camera-processing GitHub repository. It contains the local single-camera implementation and the multi-camera implementation used for the one-, two-, and three-camera experiments, together with the stress tests for 5, 10, and 15 virtual cameras.

3 Part 1: Local work

3.1 Completed tasks

Task 1.1. Webcam identification. A fixed public webcam showing Times Square in New York was selected from EarthCam. The scene contains pedestrians and vehicles at different distances, with frequent occlusion and large variations in object size.

Task 1.2. Periodic frame capture. A local Python process was implemented to retrieve the EarthCam HLS stream URL and invoke `ffmpeg` to capture one JPEG frame every five seconds. The final local experiment ran for 300 seconds and captured 60 frames of unchanged image size: 1280 pixels.

Task 1.3. Local object counting and visualisation. Ultralytics YOLOv8n was loaded on the local computer. For each frame, the program counted objects in the `person` class and, as an extension, the `bicycle`, `car`, `motorcycle`, `bus`, and `truck` classes with confidence threshold = 0.30. The timestamp, frame number, counts, inference time, and JPEG size were written to a CSV file. Bounding boxes were drawn on saved frames and a time-series graph was generated from the CSV data.

3.2 Local hardware environment

The local experiment was executed on the laptop described in Table 1. YOLOv8n inference ran on the CPU without GPU acceleration.

Table 1: Hardware and operating-system configuration used for local processing.

Parameter	Value
Computer	MacBook Air
Processor	Apple M1
CPU cores	8 (4 performance and 4 efficiency)
Memory	8 GB unified memory
Architecture	ARM64
Operating system	macOS 26.5.2
Kernel	Darwin 25.5.0

3.3 Local architecture

The local processing pipeline was implemented as follows:

EarthCam HLS stream → `ffmpeg` frame capture → YOLOv8n inference → CSV, annotated frame, and graph

3.4 Local results

The local run produced the results shown in Table 2. PyTorch and the model backend initialised during the first inference, making it slower. After this warm-up, inference remained stable and well below the five-second capture interval.

Table 2: Summary of local experiment.

Metric	Result
Average people per frame	13.68
People range	3–23
Average vehicles per frame	2.23
Vehicle range	1–6
Average YOLO time, including first frame	371.78 ms
Steady-state YOLO time, excluding first frame	357.00 ms
First-frame YOLO time	1244 ms

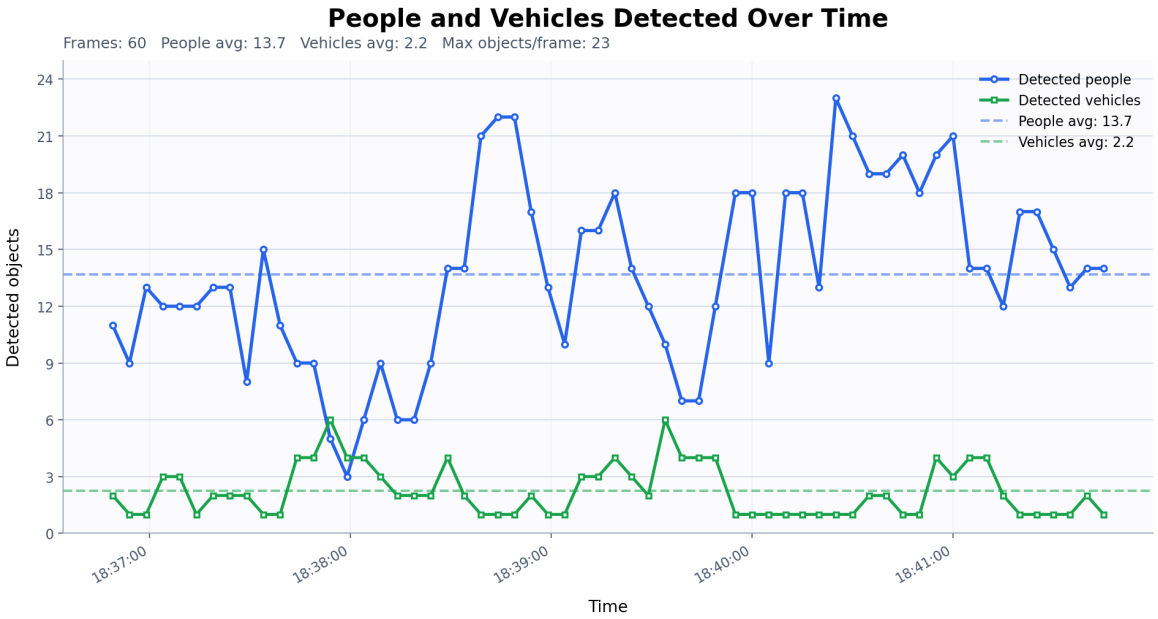


Figure 1: People and vehicles detected during local run.

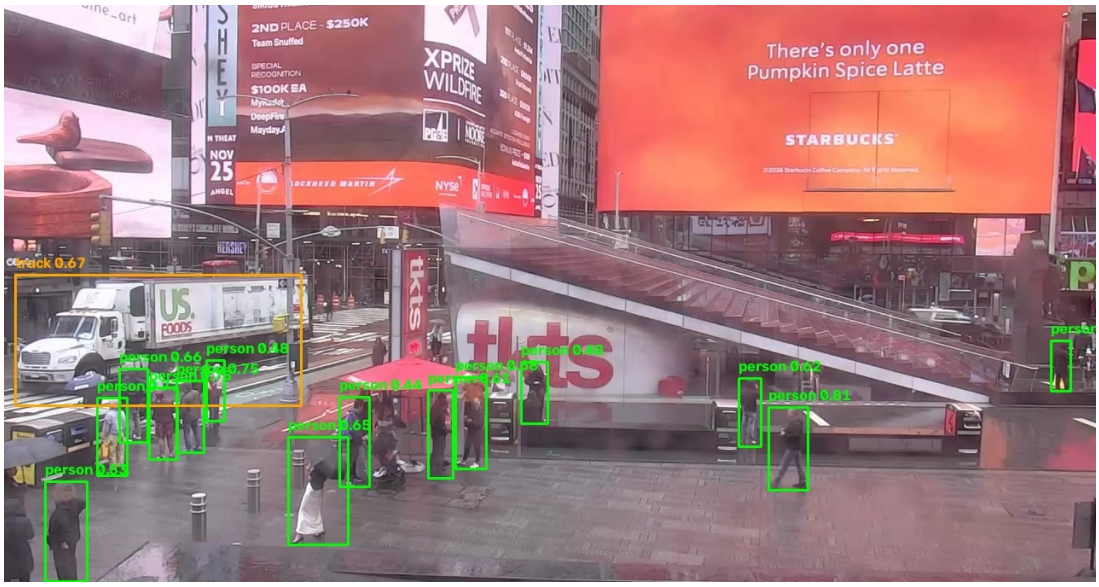


Figure 2: Example of a locally processed Times Square frame.

3.5 Detection-quality limitations

Initial experiments showed poor recall for distant pedestrians. The confidence threshold was therefore lowered to 0.30 and the inference size was not compressed for the local experiment. This improved the number of retained detections, but the quality remained limited in dense scenes. YOLOv8n often failed to separate people standing close together in a crowd. It also missed small, distant, partially occluded, or seated people. Reflections, rain, glare, and low contrast created additional missed detections.

4 Part 2: Outsourcing processing to Azure virtual machine

4.1 Completed tasks

Task 2.1. Azure VM creation and software installation. An Azure virtual machine was created and configured with the Python and computer-vision packages required to run the YOLOv8 detection service.

Task 2.2. Local capture and transmission to the VM. Frames were captured on the local computer and sent as JPEG request bodies to the detection service running on the VM. The inference resolution was halved from 1280 to 640 pixels because processing at 1280 pixels exceeded the timing window and exhausted the small VM's memory, making the detector unresponsive.

Task 2.3. Remote detection and CSV storage. YOLOv8 inference was executed on the VM. The VM returned object counts, bounding boxes, and server-side timing data as JSON. The local client combined these values with capture and network timing data and stored in a CSV file.

Task 2.4. Performance measurement. CPU usage, memory usage, network traffic, response time, and detection output were collected for the VM execution. Monetary cost was documented using Azure personal account.

4.2 Virtual machine processing architecture

The VM pipeline was implemented as follows:

EarthCam HLS stream over HTTPS → local `ffmpeg` frame capture → JPEG frame → `HTTP POST /detect` with `Content-Type: image/jpeg` → SSH local port forwarding to `127.0.0.1:8000` on the Azure VM → Python `ThreadingHTTPServer` detection service → YOLOv8n inference → `application/json` response through the same HTTP/SSH path → local CSV files, annotated frames, system metrics, and dashboard

4.3 Virtual machine configuration and setup

The deployed resource was configured as shown in Table 3.

Table 3: Azure virtual-machine configuration used in the experiments.

Property	Configuration
Resource group	`TOMA-GROUP-ARC`
Virtual-machine name	`toma-VM`
VM size	`Standard_B2ts_v2`
Operating system	Ubuntu 24.04 LTS
Usable memory observed by Linux	886 MiB
Swap	Disabled (0 B)
Remote user	`azureuser`
Authentication	SSH public-key + private key on local computer
Detection endpoint	`127.0.0.1:8000`; accessed through SSH tunnel
Python environment	Project-specific `venv`
Main packages	Ultralytics (PyTorch included), OpenCV, NumPy

The virtual environment for the detection service was created as follows:

```
sudo apt update
sudo apt install -y python3.12-venv python3-pip ffmpeg
cd /home/azureuser/vm_processing
python3 -m venv .venv
source .venv/bin/activate
pip install ultralytics opencv-python numpy --no-cache-dir
```

Ultralytics provides the YOLOv8 implementation and automatically installs PyTorch and other required dependencies. OpenCV decodes the received JPEG images, while NumPy represents them as arrays.

The detection server files, `detector_server.py` and `yolov8n.pt` containing the model weights, were transferred via SSH to the `/home/azureuser/vm_processing` directory. `detector_server.py` is a small HTTP service implemented using Python's ThreadingHTTPServer class. The service provides three endpoints:

- `GET /health` checks the availability of the service and the model;
- `POST /detect` receives a JPEG frame, runs YOLOv8, and returns a JSON response containing the detected object counts, bounding-box coordinates, queue time, inference time, and total processing time on the VM;
- `GET /metrics` returns CPU usage, memory usage, detector-process memory usage, and network counters read from the Linux `/proc` filesystem.

The service accepts connections only at `127.0.0.1:8000` and is not accessible from the public Internet. An SSH tunnel connects port `8000` on the local computer to port `8000` on the VM:

```
ssh -L 8000:127.0.0.1:8000 azureuser@<VM_PUBLIC_IP>
```

Inside this SSH session, the detector is started as follows:

```
cd /home/azureuser/vm_processing
source .venv/bin/activate
export YOLO_CONFIG_DIR=/home/azureuser/vm_processing/.cache
python detector_server.py --host 127.0.0.1 --port 8000
```

The local client is implemented in the `grabber.py` script. It obtains the addresses of the configured EarthCam streams, starts a separate worker thread for each camera, and uses the local FFmpeg installation to capture one JPEG frame every five seconds. The script then sends the JPEG bytes to `http://127.0.0.1:8000/detect`, and the SSH tunnel forwards each request to the VM.

After inference, the VM returns a JSON response containing the object counts, bounding-box coordinates, and measured processing times. The local `grabber.py` script uses this response to draw the bounding boxes and save the CSV data, processed images, VM metrics, run summary, and dashboard files. These files and `grabber.py` itself remain on the local computer and are not transferred to the VM. Thus, frame acquisition and experiment management are performed locally, while the computationally intensive YOLO inference is outsourced to Azure.

The multithreaded server can accept multiple requests simultaneously; however, there is only one instance of the YOLO model. Therefore, only one inference is performed at a time, while the remaining requests wait in the queue.

4.4 Measured timing and resource metrics

The timing values shown in Figure 3 were recorded separately:

- `vm_inference_ms` — YOLO inference time;
- `vm_total_ms` — all processing inside the VM request handler;

- `network_overhead_ms` — VM round-trip time minus VM processing time;
- `vm_roundtrip_ms` — time from sending the request to receiving the response;
- `capture_ms` — time required to obtain one frame from EarthCam;
- `end_to_end_ms` — complete capture, transmission, and processing time.

CPU and memory are sampled once per second from the VM. Bandwidth consumption is calculated as the change in the VM network byte counters between the beginning and end of each run. This includes traffic observed by the VM during the measurement window, so unrelated background traffic may contribute a small amount.

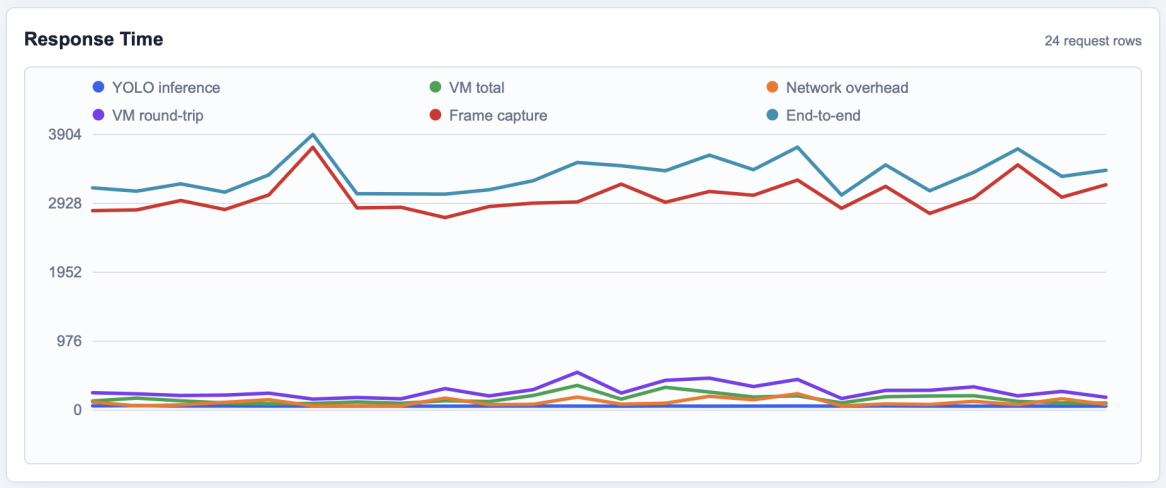


Figure 3: Timing components measured for every frame in the single-camera VM run.

4.5 Single-camera VM results

The single-camera experiment used the Times Square stream for 120 seconds, with one frame scheduled every five seconds (24 frames in total), an input size of 640 pixels, and a confidence threshold of 0.30.

Table 4 summarises the detection, timing, and VM resource measurements. The average YOLO inference time on the VM was 58.50 ms, while the average VM round-trip time was 274.08 ms. The complete end-to-end time was substantially higher because obtaining a frame from the external EarthCam stream took approximately three seconds. Thus, for this experiment, frame acquisition rather than inference was the dominant part of the complete processing cycle.

Table 4: Summary of the single-camera VM experiment.

Metric	Average	Maximum or total
People per frame	10.25	15
Vehicles per frame	1.54	4
Frame capture time	3021.38 ms	3723 ms
YOLO inference time	58.50 ms	65 ms
Total processing inside the VM	163.08 ms	351 ms
Network and client overhead	111.00 ms	233 ms
VM round-trip time	274.08 ms	537 ms
End-to-end time	3326.38 ms	3904 ms
CPU usage	0.86%	4.11%
Memory usage	78.90%	80.25%
Network traffic, received / sent	—	11.05 MB / 0.79 MB

Figure 4 shows that the number of detected people varied between 5 and 15, while the number of detected vehicles varied between 0 and 4.

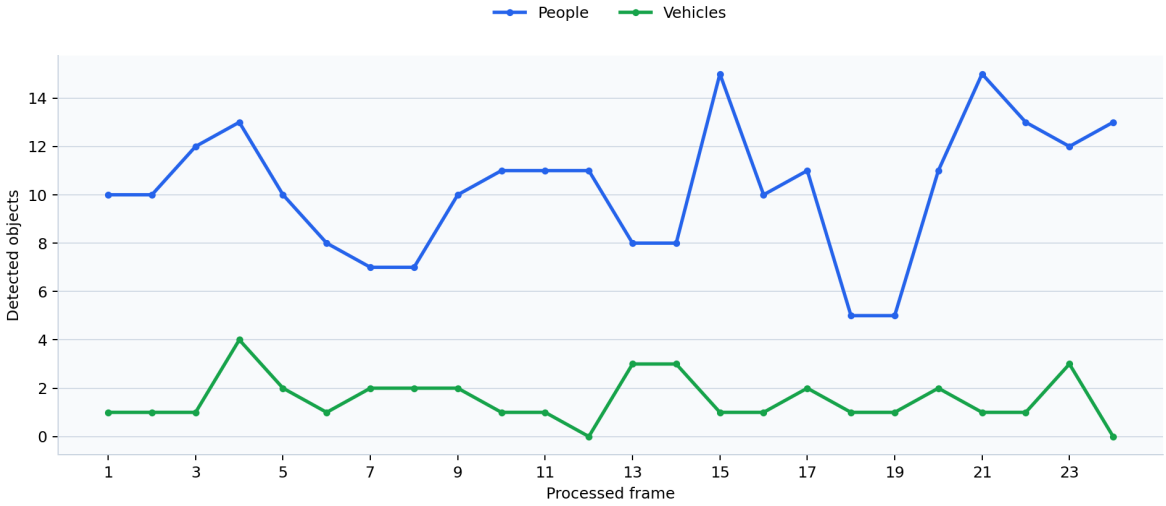


Figure 4: People and vehicles detected during the single-camera VM run.

Figure 5 presents the once-per-second VM samples. Memory usage remained close to 79%, whereas CPU usage was low between requests and rose briefly during frame processing.

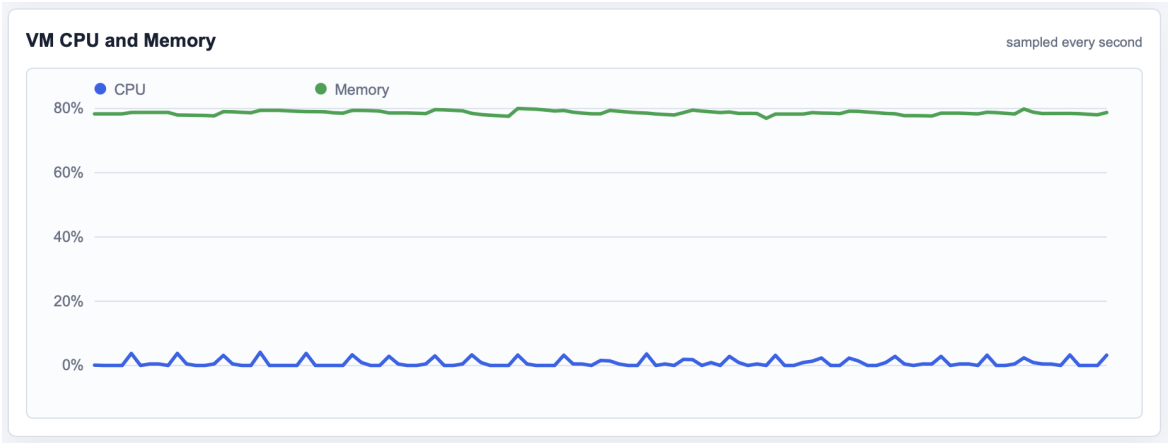


Figure 5: CPU and memory usage during the single-camera VM run.

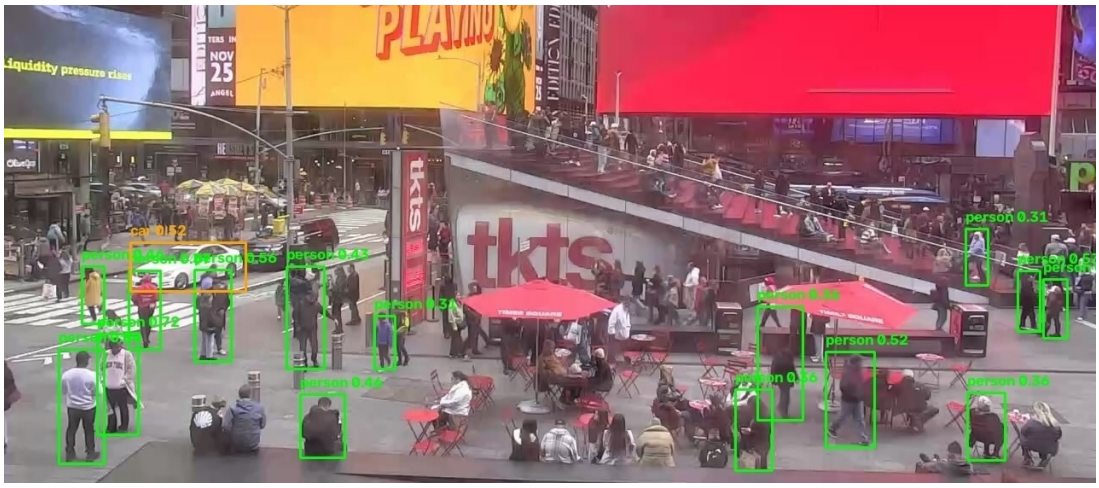


Figure 6: Example of a frame processed by YOLOv8n on the Azure VM.

5 Part 3: Serverless processing in Azure

5.1 Completed tasks

Task 3.1. Static web page. A static website was created in the `$web` container of the `tomastorage` Storage Account. It displays detection counts, response times, request statuses, and the latest processed images.

Task 3.2. Local frame capture and cloud upload. Frames were captured locally from the EarthCam HLS stream using FFmpeg and uploaded every five seconds to the `raw-images` container. This input container is separate from the `$web` container used by the static website.

Task 3.3. Blob-triggered serverless application. The `ProcessRawImage` blob-trigger Function was deployed to the `toma-app2026` Function App. Event Grid invokes the Function only when a new image is uploaded to the `raw-images` container.

Task 3.4. Azure AI. An Azure AI Vision service was configured to analyse the uploaded images using the People and Objects features of the Image Analysis API.

Task 3.5. Serverless image processing. The Function sends every raw image to Azure AI Vision, counts detected people and vehicles, and draws bounding boxes around the detected objects. Raw images, annotated images, JSON results, CSV measurements, and the latest processed frame are saved in the website Storage Account.

Task 3.6. Performance measurement. Frame-capture time, upload time, Azure AI processing time, Function processing time, end-to-end time, and processing errors were recorded. Detection efficiency and monetary cost are evaluated separately together with the results of the other architectures.

5.2 Serverless processing architecture

The serverless processing pipeline was implemented as follows:

EarthCam HLS stream over HTTPS → local `ffmpeg` frame capture → HTTPS Blob upload through the Azure Storage SDK to `raw-images` → `BlobCreated` notification through Event Grid → Event Grid-based `ProcessRawImage` Blob Trigger → HTTPS REST `POST` with `application/octet-stream` to Azure AI Vision → JSON response with object counts and bounding boxes → HTTPS Blob writes through the Azure Storage SDK to `$web` → browser HTTPS requests for the static dashboard, CSV, JSON, and annotated images

5.3 Azure app configuration and setup

The serverless application, whose configuration is summarised in Table 5, was deployed to the `toma-app2026` Function App using Python 3.11 and the Linux Flex Consumption hosting model. Input images were uploaded to the `raw-images` container in the `tomagrouparcb131` Storage Account. The static website and processing results were stored in the `$web` container of a separate Storage Account, `tomastorage`.

The Event Grid subscription reacted to `BlobCreated` events in the input container and forwarded them to the `ProcessRawImage` Function. Separating the input and output storage prevented recursive Function executions when annotated images were saved.

Table 5: Azure serverless application configuration.

Parameter	Value
Function App	`toma-app2026`
Function	`ProcessRawImage`
Runtime	Python 3.11
Hosting model	Linux Flex Consumption
Trigger	Event Grid blob trigger
Input Storage Account	`tomagrouparcb131`
Input container	`raw-images`
Website Storage Account	`tomastorage`
Website container	`$web`
Azure AI API version	`2024-02-01`
Confidence threshold	0.20

Images were analysed using version `2024-02-01` of the Azure AI Vision Image Analysis API with the `people` and `objects` features. A confidence threshold of 0.20 was used for both people and vehicles. Azure AI Vision does not expose the YOLO `imgsz=640` parameter; therefore, no such parameter was applied, and the input images were sent at their captured resolution without additional resizing.

The local `grabber.py` process created an independent worker for each camera. After frame capture, each JPEG image was uploaded to Blob Storage, and processing began asynchronously after the corresponding event was delivered to Azure Function.

The Function sent the image to Azure AI Vision and extracted people from `peopleResult` and vehicles from `objectsResult` and drew the bounding boxes.

The accumulated `runs/results.csv` table was used by the dashboard to select and display individual experimental runs. The page presented people and vehicle counts by camera, Azure AI time, Function time, end-to-end time, request statuses, and the latest annotated frame from each camera.

5.4 Measured timing and resource metrics

The following timing metrics were recorded:

- `vision_ms` — time spent waiting for the Azure AI Vision response;
- `response_time_ms` — time spent executing the main part of the Azure Function;
- `queue_delay_ms` — delay caused by image upload and blob trigger activation;
- `end_to_end_ms` — time from obtaining the completed JPEG frame to preparing the cloud-processing result;
- `capture_ms` — time required to obtain one frame from EarthCam.

5.5 Single-camera serverless experiment

The single-camera experiment summarised in Table 6 used the Times Square stream. All 24 images were uploaded and processed during the run lasted 120 seconds, with one frame captured every five seconds.

Table 6: Summary of the single-camera serverless experiment.

Metric	Average	Maximum or total
Frames processed	—	24/24
Upload errors	—	0
Processing errors	—	0
People per frame	8.88	5–12
Vehicles per frame	0.17	0–1
Frame capture time	2970.46 ms	3519 ms
Blob upload time	171.79 ms	496 ms
Azure AI Vision time	520.75 ms	916 ms
Function processing time	567.42 ms	964 ms
Queue and trigger delay	806.12 ms	5890 ms
Serverless end-to-end time	1373.54 ms	6692 ms
Complete time including capture	4344.00 ms	9795 ms
Uploaded raw image data	—	4.11 MB

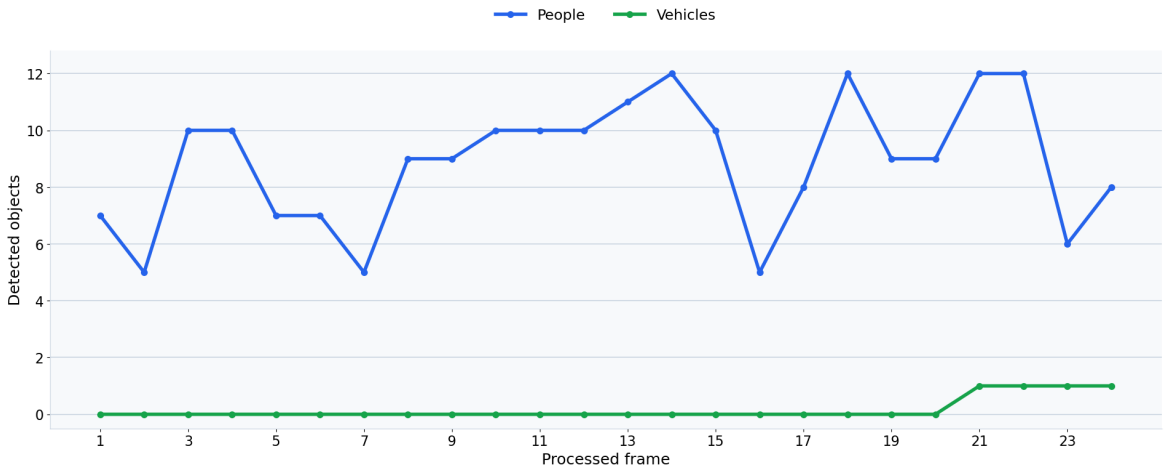


Figure 7: People and vehicles detected during the single-camera serverless run.

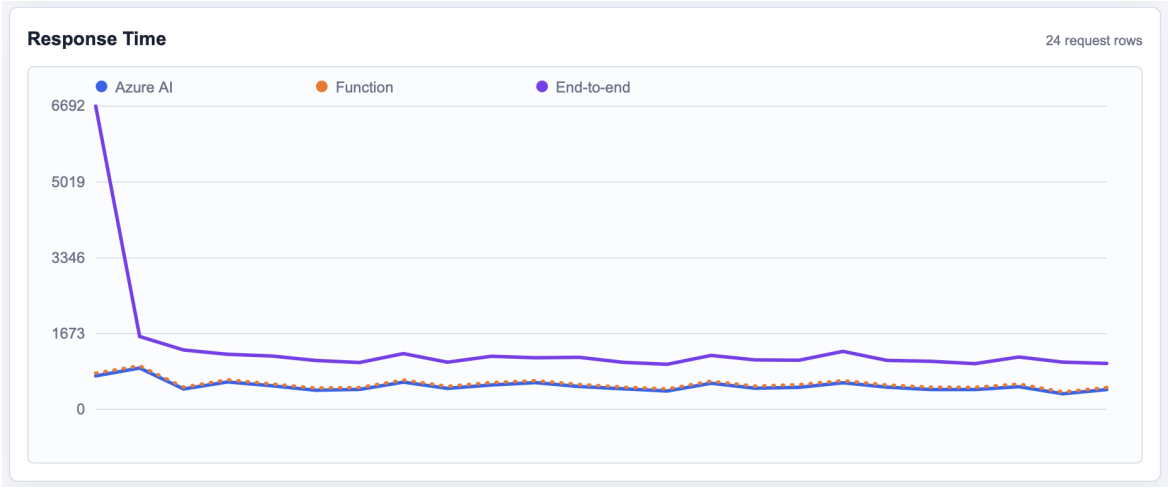


Figure 8: Timing components measured for every frame in the single-camera serverless run.

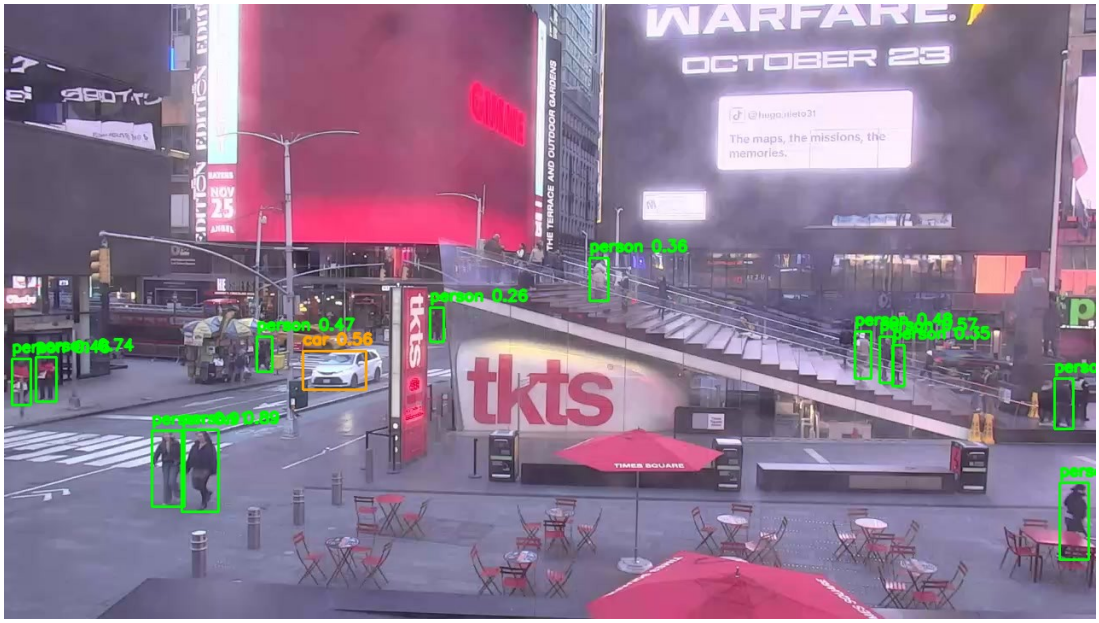


Figure 9: Example of a frame processed by Azure AI Vision.

Azure AI Vision accounted for most of the Function execution time: on average, 520.75 ms out of 567.42 ms. The remaining processing inside the Function, including response parsing and bounding-box drawing, took approximately 47 ms.

The largest delay was observed for the first image. Its queue and trigger delay was 5890 ms, while its serverless end-to-end time was 6692 ms. This corresponds to the initial cold start of the Function App. For subsequent frames, the serverless end-to-end time was approximately between 1.0 and 1.6 seconds.

The average complete processing cycle, including HLS frame capture, was 4.34 seconds. Therefore, the average processing time remained within the five-second interval, although individual frames occasionally exceeded it.

6 Part 4: Multi-camera management

6.1 Completed tasks

Task 4.1. Increasing number of webcams. Three fixed EarthCam sources were configured: Times Square, Abbey Road, and Liberty Bell. Each camera has an independent local worker and requests one frame every five seconds. The workers start together to create simultaneous load.

Task 4.2. Multi-camera VM experiment. Frames from one, two, and three cameras were sent to the VM created in Part 2. Each run lasted 120 seconds, with one frame captured every five seconds per camera, producing 24 frames per camera. The experiments used a confidence threshold of 0.30 and an inference image size of 640 pixels. Response time, queue time, CPU usage, memory usage, and network byte counters were recorded. The same warmed VM server remained running across the final one-, two-, and three-camera runs. A separate stress test replayed three saved raw JPEG frames as 5, 10, and 15 simultaneous virtual cameras, without the approximately three-second EarthCam capture delay. All virtual cameras started simultaneously, sent one frame every five seconds, and completed three request batches per run.

Task 4.3. Multi-camera serverless experiment. Frames from one, two, and three cameras were uploaded to the serverless application created in Part 3. Every run lasted 120 seconds, and each camera produced 24 frames at five-second intervals. Timing data, processing results, and

Azure AI throttling errors were recorded to identify the largest stable camera configuration that is 2-camera system.

6.2 Virtual machine results for 1-3 cameras

The average YOLO inference time remained approximately 59 ms for all three configurations. Queue time increased as simultaneous requests were added, but the maximum observed queue of 111 ms remained far below the five-second interval. CPU and memory use also increased, but all 72 requests in the three-camera run were processed successfully and on time. Table 7 compares timing and resource use.

Table 7: VM timing and resource results at `imgsz=640`.

Cameras	Frames completed	Infer. avg (ms)	Round-trip avg (ms)	End-to-end avg (ms)	CPU avg/max (%)	RAM avg/max (%)
1	24/24	58.50	274.08	3326.38	0.86/4.11	78.90/80.25
2	48/48	59.50	269.23	3202.88	1.61/8.13	81.72/84.37
3	72/72	59.19	352.86	3262.94	2.37/11.68	82.85/85.37

The end-to-end time was approximately 3.2–3.3 seconds, but this value was dominated by frame capture from the external EarthCam streams. Average capture time alone was 3021 ms for one camera, 2905 ms for two cameras, and 2883 ms for three cameras. It is therefore important not to interpret the 3–4 second line in the dashboard as YOLO processing time or as proof that the VM has reached its capacity.

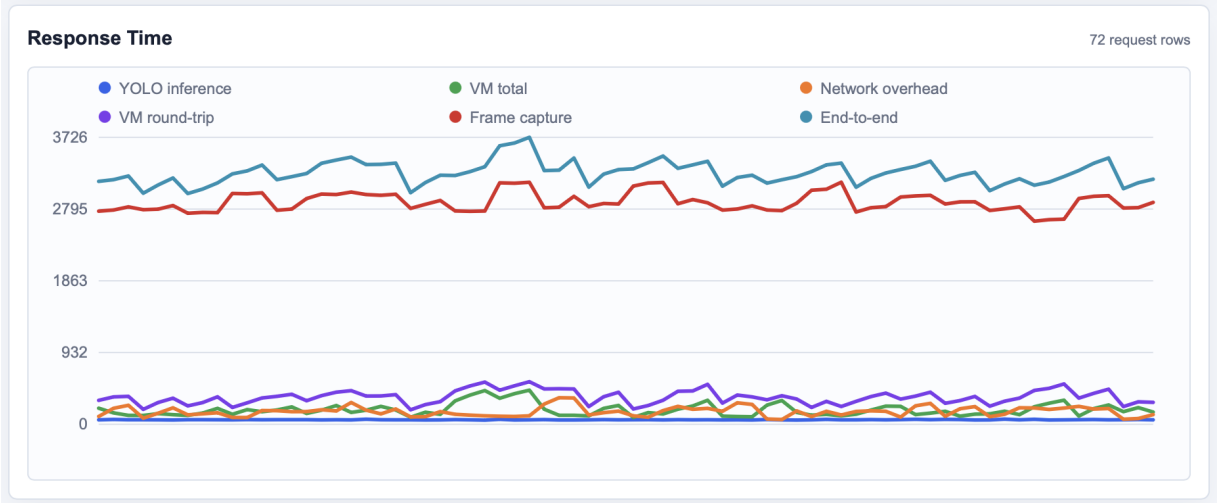


Figure 10: Response-time components for all 72 requests in the three-camera VM run.

Table 8 shows that the request queue increased gradually as additional cameras were introduced. No queue was observed with one camera. For two cameras, the average queue time was 9.94 ms and the maximum was 56 ms; for three cameras, these values increased to 20.61 ms and 111 ms, respectively. This increase is expected because the cameras send frames simultaneously, while access to the YOLO model is serialised on the VM. Nevertheless, the maximum queue time remained much lower than the five-second frame interval. Therefore, the VM was not overloaded by the one-, two-, or three-camera configurations, and all scheduled requests were processed successfully.

Incoming network traffic increased from 11.05 MB for one camera to 31.14 MB for three cameras, reflecting the larger number of JPEG frames transmitted to the VM. Outgoing traffic was considerably lower because the VM returned only compact JSON responses containing detection

counts and timing information. The traffic did not increase in perfectly proportional steps because JPEG frame sizes varied between the camera streams and the VM network counters could also include a small amount of background traffic.

Table 8: VM queue and network results at `imgsz=640`.

Cameras	Queue avg. (ms)	Queue max. (ms)	Network in (MB)	Network out (MB)
1	0.00	0	11.05	0.79
2	9.94	56	19.12	1.03
3	20.61	111	31.14	1.31

Figures 11–13 show the dashboard results for the complete three-camera run. The dashboard separates detections by camera, while the timing and resource graphs combine all 72 requests in the same two-minute measurement window.

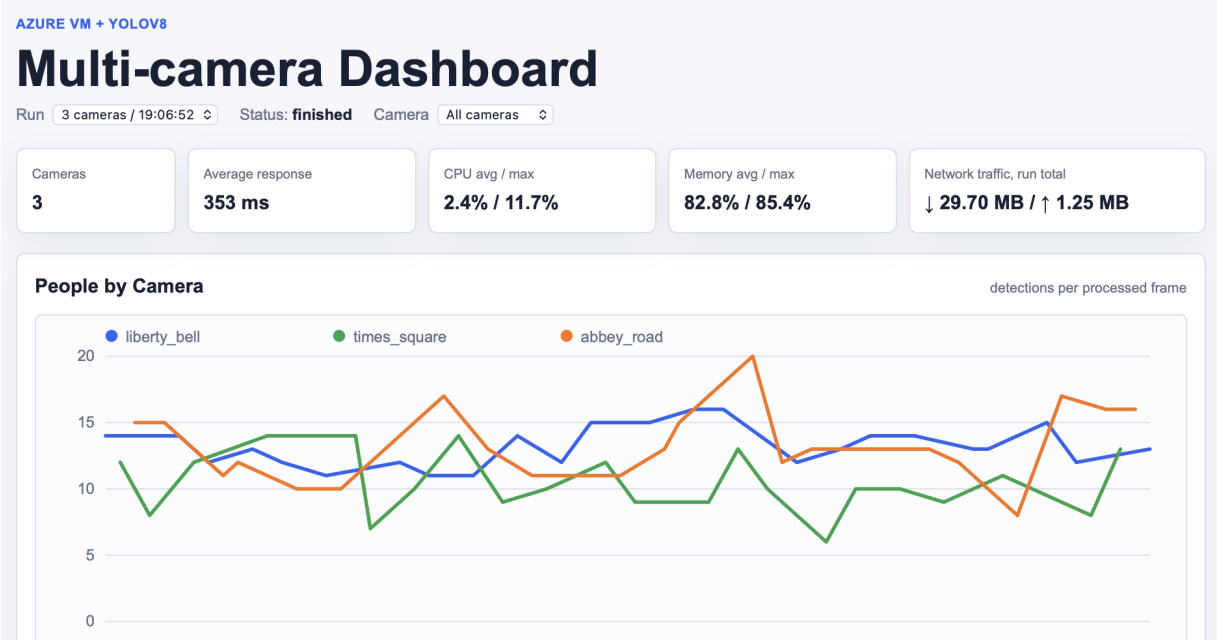


Figure 11: Three-camera VM dashboard summary and people detections by camera.

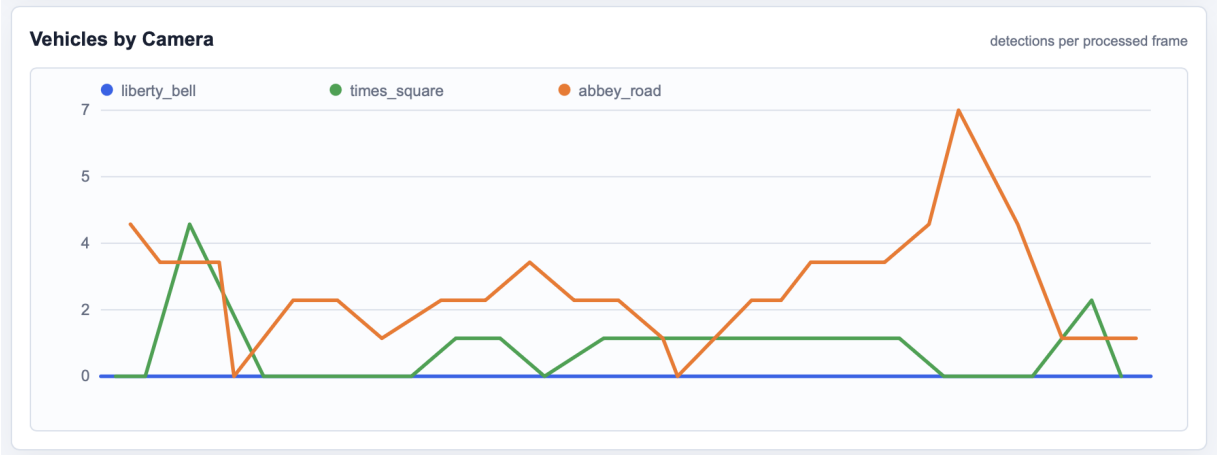


Figure 12: Vehicle detections separated by camera during the three-camera VM run.



Figure 13: VM CPU and memory usage together with the latest processed frame from each camera.

6.3 VM processing stress test

The synthetic VM stress-test pipeline was implemented as follows:

> Three JPEG frames saved during the real-camera experiment → assignment to 5, 10, or 15 virtual cameras → simultaneous `HTTP POST /detect` requests with `Content-Type: image/jpeg` every five seconds → SSH local port forwarding to `127.0.0.1:8000` on the Azure VM → Python `ThreadingHTTPServer` request handling → request queue on the Azure VM → serialised YOLOv8n inference → `application/json` responses through the same HTTP/SSH path → CSV result tables

The three real webcams did not reach the VM limit: all requests were processed successfully and the maximum queue time was only 111 ms. To increase only the load sent to the VM, a separate script replayed the three saved raw JPEG frames as additional virtual cameras. The images were assigned to virtual cameras in a round-robin order. No new EarthCam frame was captured during this test, so the result reflects VM request handling, JPEG decoding, YOLO inference, and network communication rather than HLS capture latency.

Table 9: Synthetic VM stress-test results at `imgsz=640`.

Virtual cameras	Completed / expected	Failed	Missed	Inference avg. (ms)	Queue avg./max. (ms)	Round-trip avg./max. (ms)
5	15/15	0	0	61.73	99.20/221	734.40/910
10	30/30	0	0	61.43	159.87/474	920.60/1567
15	18/45	15	12	61.33	128.33/382	1118.22/1817

Table 10: Resource consumption during the synthetic VM stress tests.

Virtual cameras	CPU avg./max. (%)	RAM avg./max. (%)	Network in/out (MB)
5	3.71/17.29	83.25/84.90	6.09/0.19
10	8.99/30.66	86.56/88.45	12.14/0.35
15	5.87/41.99	93.12/96.18	13.42/0.38

Five and ten virtual cameras completed every expected request without failures, missed requests, or five-second deadline violations. With 15 simultaneous virtual cameras, only 18 of 45 expected requests completed successfully, 15 requests timed out, and 12 requests were never attempted before the scheduling window ended. Average memory use rose to 93.12% and reached a maximum of 96.18%. CPU use remained comparatively low, showing that RAM pressure, rather than CPU saturation, was the principal limiting resource.

Therefore, the largest stable load demonstrated in this experiment was **10 simultaneous virtual cameras**. The system was overloaded at 15 cameras. The exact transition point between 10 and 15 cameras was not measured further.

6.4 Serverless multi-camera results for 1–3 cameras

The Times Square, Abbey Road, and Liberty Bell streams were used for the comparison. All source images were successfully captured and uploaded to Blob Storage. Errors occurred only during Azure AI Vision processing in the three-camera run.

Figure 14 and Table 11 presents the timing and processing results. Frame-capture and complete-pipeline averages and maxima were calculated over successfully processed frames. Complete time includes both HLS frame capture and serverless end-to-end processing.

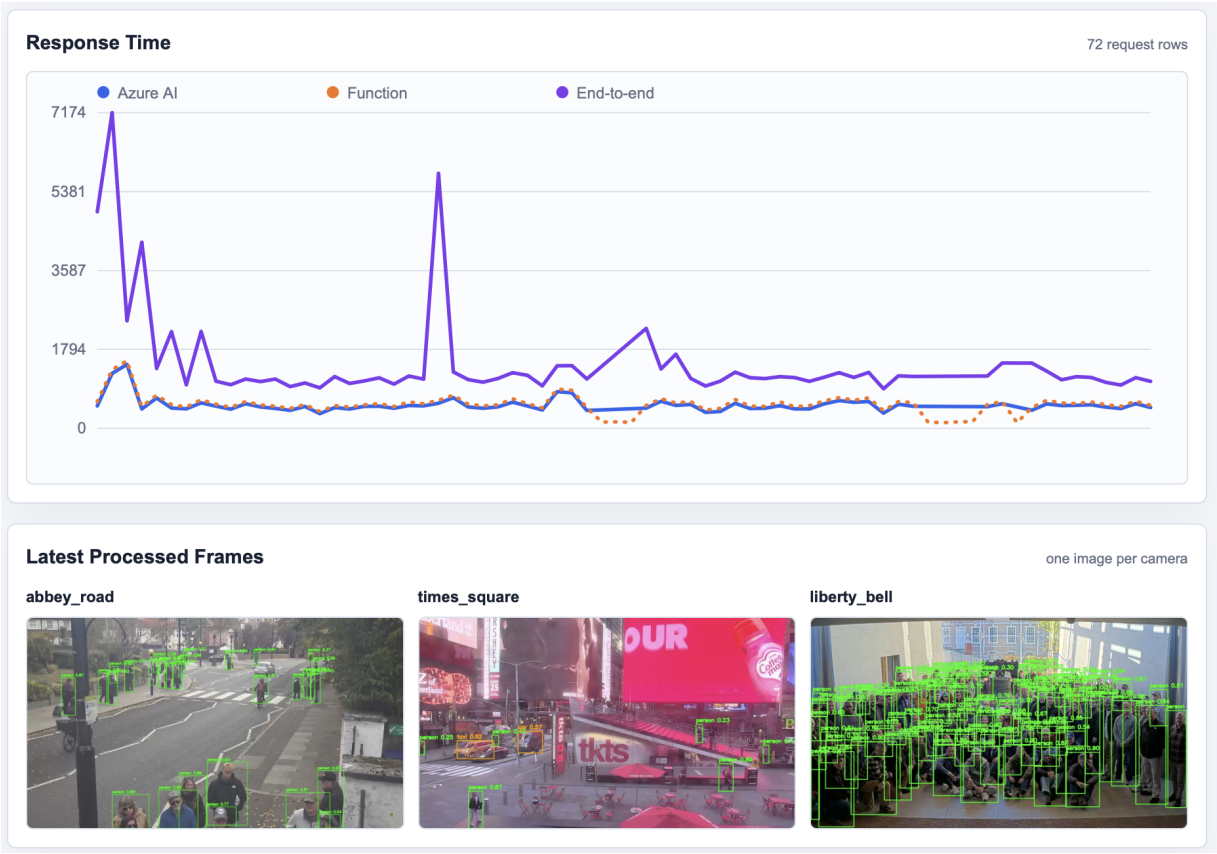


Figure 14: Response times and latest annotated frames from the three-camera serverless experiment.

Table 11: Serverless timing results for one, two, and three cameras.

Cameras	Expected	Processed	Errors	Frame capture avg./max. (ms)	Azure AI avg./max. (ms)	Function avg./max. (ms)	Serverless end-to-end avg./max. (ms)	Complete time avg./max. (ms)
1	24	24	0	2970.46 / 3519	520.75 / 916	567.42 / 964	1373.54 / 6692	4344.00 / 9795
2	48	48	0	2948.44 / 3290	800.96 / 1858	854.50 / 1922	1506.23 / 5506	4454.67 / 8331
3	72	64	8	2900.53 / 3499	535.45 / 1452	594.31 / 1536	1499.70 / 7174	4400.23 / 10104

14:19:43	liberty_bell	13	0	0	2191 ms	ok
14:19:43	times_square	13	15	0	1650 ms	ok
14:19:38	liberty_bell	12	-	-	-	error
14:19:37	abbey_road	12	26	1	1162 ms	ok
14:19:37	times_square	12	8	0	1122 ms	ok
14:19:35	abbey_road	11	-	-	-	error
14:19:35	liberty_bell	11	-	-	-	error
14:19:35	times_square	11	-	-	-	error
14:19:28	liberty_bell	10	0	0	2178 ms	ok

Figure 15: Failed requests from three-camera serverless experiment.

The one- and two-camera runs completed without errors. With three cameras, several frames repeatedly received an `HTTP 429 Too Many Requests` response from Azure AI Vision. An additional three-camera run in `staggered` mode distributed the requests across the five-second interval but did not eliminate the errors.

Therefore, changing the request timing did not resolve the problem. The limitation occurred at the Azure AI Vision level rather than during frame capture, Blob Storage upload, or Azure Function execution. The largest successful stable load demonstrated in the experiment was two cameras.

7 Discussion

7.1 Problems encountered and solutions

7.1.1 RAM exhaustion at high inference resolution

The first VM experiments attempted to use an inference size of 1280 pixels, matching the local quality-oriented configuration. On the small VM, this caused severe memory pressure. A two-minute run attempted only two requests: one succeeded and one timed out, while 22 expected requests were missed. Maximum reported memory use reached 95.26%.

After this run, even 640-pixel requests temporarily required approximately 10–12 seconds. The problem was diagnosed with `vmstat`: only 4–20 MiB of memory remained free, the VM had no swap, disk input rose to approximately 118–136 MB/s, and CPU I/O wait reached 47–63%. The CPU was not fully occupied; it was waiting while memory pages were repeatedly read from disk. Stopping the long-lived detector process increased available memory to 628 MiB.

The detector was restarted, warmed with a preliminary 640-pixel run, and then kept at 640 pixels for the complete 1-, 2-, and 3-camera comparison. Performance returned to the expected range.

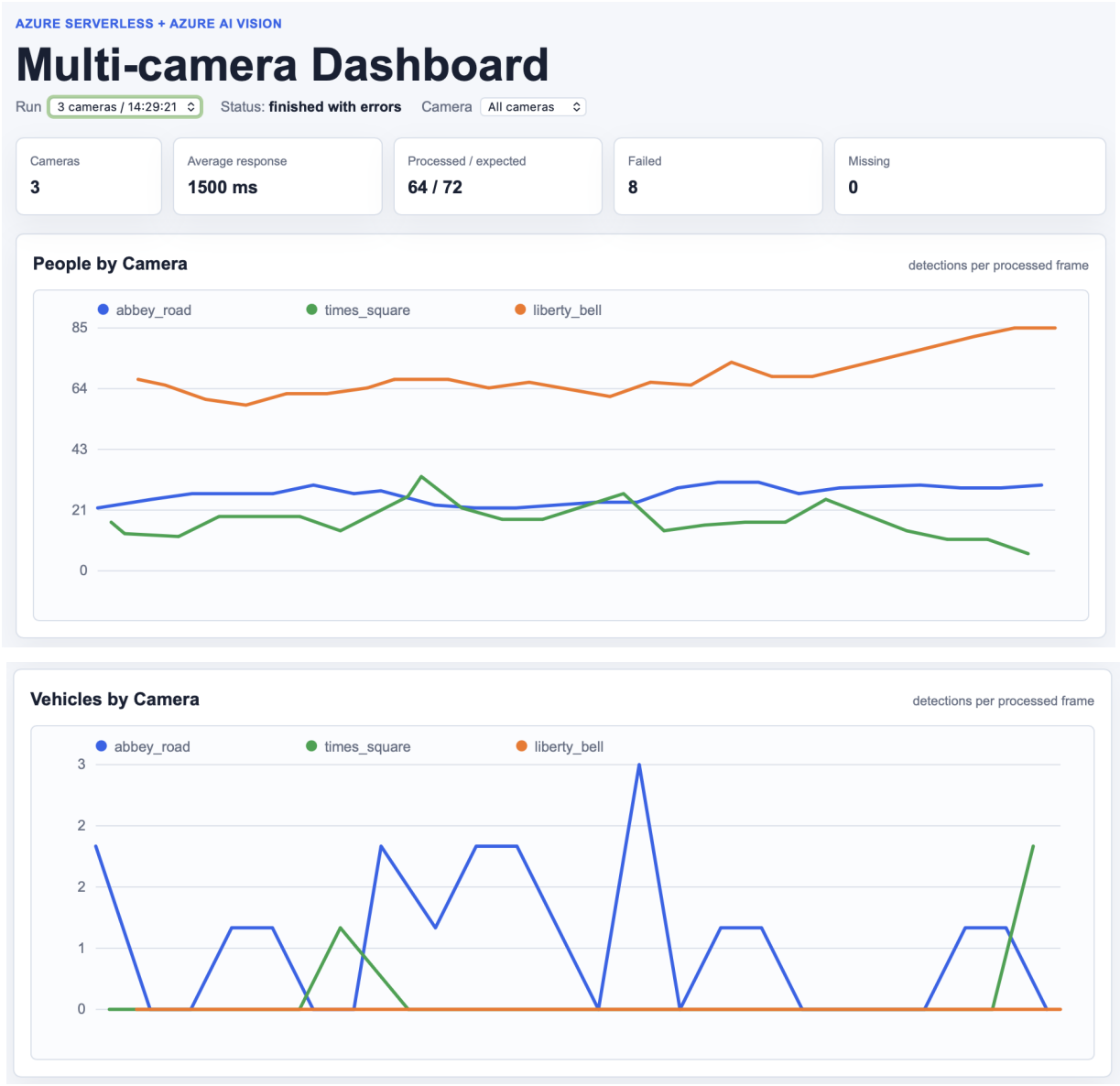


Figure 16: People and vehicle counts displayed by the serverless dashboard for the three-camera experiment.

7.1.2 Separating capture delay from VM performance

It was found that the main bottleneck of the system was not video processing, but frame capture from the stream. EarthCam uses HLS, which delivers video as a sequence of segments rather than individual images. For each capture, FFmpeg must connect to the stream, retrieve a segment, decode it, and extract a JPEG frame; this takes about three seconds, whereas YOLO inference on the VM takes only about 59 ms.

Therefore, the complete pipeline using real EarthCam streams was experimentally tested with one, two, and three cameras and remained within the five-second interval per frame. An additional experiment reused previously captured frames to simulate simultaneous loads from five and ten cameras, both of which were processed successfully by the isolated VM service. However, with 15 simultaneous virtual cameras, timeouts and missed requests occurred. Consequently, ten cameras represent a confirmed supported load for the VM service, but not a confirmed capacity of the complete pipeline with ten real HLS streams.

7.2 Experiments comparison

The available timing results for the local, VM, and serverless experiments are summarised in Table 12. The local experiment measured only YOLO inference time, which was higher than on the VM but lower than Azure AI Vision processing time.

In both Azure experiments, the largest component of the complete pipeline time was the approximately three-second HLS frame-capture delay. YOLO inference in the VM experiments remained fast at approximately 59 ms. Azure AI Vision processing was approximately 9–14 times slower than YOLO inference on the VM. However, the VM round trip included not only inference but also image decoding, queueing, network communication, and request processing on the VM. The serverless cloud path additionally included image upload, Event Grid delivery, Azure Function activation, and Azure AI Vision processing. Consequently, when the complete cloud paths are compared, the difference decreases to approximately 4–6 times in favour of the faster VM architecture.

Table 12: Average timing in milliseconds for the local, VM, and serverless experiments.

Architecture	Cameras	Successful	Schedule lag	Capture	Upload	Decode	AI / model	Queue / trigger	Service total	Network overhead	Cloud path	Complete pipeline
Local	1	60/60	—	—	—	—	**371.78**	—	**371.78**	—	—	—
VM	1	24/24	4.25	3021.38	—	5.12	**58.50**	**0.00**	**163.08**	111.00	**274.08**	**3326.38**
VM	2	48/48	5.94	2905.02	—	8.54	**59.50**	**9.94**	**146.73**	122.50	**269.23**	**3202.88**
VM	3	72/72	7.82	2883.43	—	9.03	**59.19**	**20.61**	**187.74**	165.12	**352.86**	**3262.94**
VM stress	5	15/15	5.27	—	—	14.67	**61.73**	**99.20**	**598.33**	136.07	**734.40**	—
VM stress	10	30/30	4.67	—	—	26.33	**61.43**	**159.87**	**600.90**	319.70	**920.60**	—
VM stress	15	18/45	6.67	—	—	22.89	**61.33**	**128.33**	**643.83**	474.39	**1118.22**	—
Serverless	1	24/24	2.33	2970.46	171.79	—	**520.75**	**806.12**	**567.42**	—	**1373.54**	**4344.00**
Serverless	2	48/48	4.83	2948.44	181.81	—	**800.96**	**651.73**	**854.50**	—	**1506.23**	**4454.67**
Serverless	3	68/72	95.41	3161.82	427.15	—	**846.85**	**919.13**	**904.32**	—	**1823.46**	**4985.28**

Table 13 summarises the largest stable load demonstrated in each experiment. The VM successfully processed three live HLS camera streams and remained stable with ten virtual cameras in the separate prerecorded-frame stress test; the 15-camera stress run was interrupted by memory pressure and request timeouts. The serverless architecture remained stable with two live cameras, while the three-camera run produced Azure AI Vision `HTTP 429 Too Many Requests` errors.

Table 13: Largest stable load demonstrated in each experiment.

Architecture	Test type	Cameras
Virtual machine	Prerecorded-frame stress test	10
Serverless	Real-camera experiment	2

Model performance was evaluated manually using the saved annotated images. Each detection was classified as a true positive or false positive, while visible objects without a bounding box were counted as false negatives. The resulting precision and recall values were combined into the F1-score, which is an appropriate metric for object-detection tasks. As shown in Table 14, vehicles were detected more reliably than people in the local and VM experiments because they were less numerous and generally appeared larger. People were more difficult to detect because they could be seated, overlap in crowds, or appear small in the background. The serverless experiment was an exception, with a substantially lower vehicle F1-score.

Table 14: F1-scores for people and vehicle detection.

Architecture	People F1-score	Vehicle F1-score
Local	62.96%	83.08%
VM	65.43%	87.18%
Serverless	66.41%	54.55%

7.3 Monetary cost

The Azure for Students subscription included free quotas and credits, which made it difficult to determine the exact cost of experiments. For the period from 23 September to 18 October 2026, Azure Cost Analysis reported an actual cost of EUR 4.30 and a forecast cost of EUR 4.35, distributed approximately as shown in Table 15.

Table 15: Actual Azure subscription cost.

Service category	Cost	Interpretation
Virtual Network	EUR 2.51	Primarily the public IP address
Storage	EUR 1.76	SSD and Storage Account charges
Virtual Machines	EUR 0.07	While running
Bandwidth	EUR 0.01	Network data transfer
Event Grid	EUR 0.00	Free
Functions	EUR 0.00	Free
Azure AI Vision / Foundry Tools	EUR 0.00	Free
No subscription ID adjustment	EUR -0.04	—
Total	**EUR 4.30**	

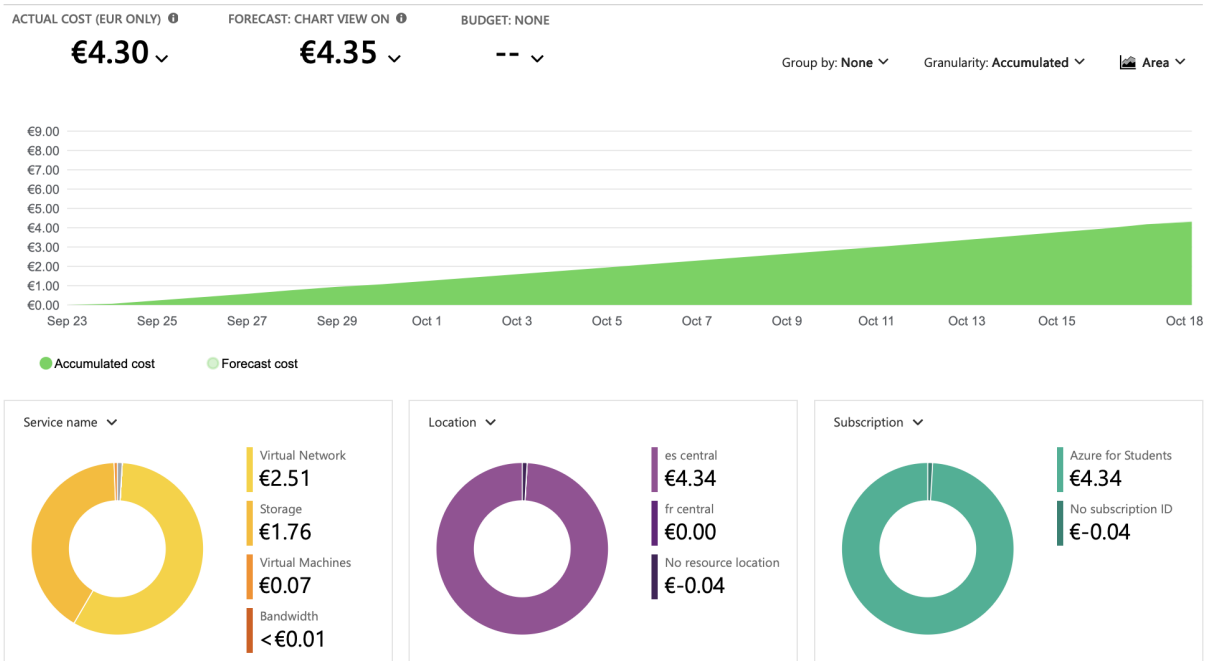


Figure 17: Azure Cost Analysis.

Most of the observed cost did not result directly from model inference. The public IP address and managed SSD continued to generate charges while the resources remained provisioned, even when no experiment was running. Based on the daily values in Azure, the approximate costs associated with each architecture are shown in Table 16.

Table 16: Approximate daily cost of the experimental architectures.

Architecture	Included charges	Estimated cost
Local	No Azure resources	EUR 0.00
VM	Public IP (EUR 0.11), managed SSD (EUR 0.07–0.08), and VM compute (EUR 0.05 for a day with VM use)	EUR 0.23–0.24 per provisioned day
Serverless	Blob Storage (EUR 0.01–0.02), Event Grid (up to EUR 0.01), and Queue v2 LRS storage (EUR 0.01). Azure Functions and Azure AI Vision marked as free	EUR 0.03–0.04 per active day

8 Conclusions

This practice implemented and compared three image-processing architectures: local YOLOv8n inference, a YOLOv8n service hosted on an Azure VM, and a serverless pipeline using Blob Storage, Event Grid, Azure Functions, and Azure AI Vision. The experiments covered single- and multi-camera processing, response time, resource use, detection quality, monetary cost, and load limits. HLS frame capture, at approximately three seconds, dominated the complete pipeline time, while the VM provided the fastest cloud-processing path and remained stable with three live cameras and ten virtual cameras. The serverless architecture had lower observed infrastructure cost and required no dedicated server management, but Azure AI Vision throttling limited the demonstrated stable load to two cameras. Overall, the experiments show that architecture selection involves a trade-off between processing speed, operational simplicity, cost, and supported concurrent load.